Análisis de charlas de seguridad

Data poisoning y jailbreaks en modelos de lenguaje

Fichas de las charlas

Charla 1

Título	Hackin' AI: Creando una IA… ¿Qué puede salir mal?
Autor	Chema Alonso
Evento y año	OpenExpo Europe / Metaworld Congress, 2025
Idioma	Español
Enlace	https://www.youtube.com/watch?v=cj3u4EtRzks
Resumen	Chema repasa, con papers y pruebas propias, cómo los modelos de IA se diseñaron para funcionar y no para ser seguros. Enseña extracción de datos, envenenamiento, alucinaciones y un jailbreak con juego de rol. Acaba avisando de los riesgos de meter IA en todos los dispositivos.
Relación	Cubre el envenenamiento de datos y el jailbreak, los dos temas de mi investigación.

Charla 2

Título	Instruction Backdoor Attacks Against Customized LLMs
Autores	Rui Zhang, Hongwei Li, Rui Wen, Wenbo Jiang, Yuan Zhang, Michael Backes, Yun Shen y Yang Zhang (la presentó otra persona en su nombre)
Evento y año	USENIX Security 2024
Idioma	Inglés
Enlace	https://www.youtube.com/watch?v=OHzoSIrJVgI

En inglés y de unos 10 minutos, la charla explica un ataque de tipo backdoor (puerta trasera) contra modelos personalizados como los GPTs de OpenAI. En vez de envenenar los datos de entrenamiento, el atacante publica una versión del modelo con instrucciones escondidas en el prompt que se activan con un disparador: una palabra, una forma de construir la frase o un tema. En las pruebas, la mayoría de los modelos pasan del 90 % de éxito, y los más potentes (GPT-3.5, GPT-4 y Claude 3) lo ponen más fácil porque siguen mejor las instrucciones. Las defensas que proponen solo funcionan en parte. Para mi tema es interesante porque se parece al data poisoning, pero manipulando las instrucciones y no los datos.

Lo que cuenta el vídeo coincide con la página oficial del artículo en USENIX: es el primer ataque de este tipo, tiene tres niveles (palabra, sintaxis y semántica) y no hace falta tocar el

modelo. Las cifras son las que da el ponente. Hay que tener en cuenta que quien presenta no es el autor, así que es un resumen del trabajo.

Charla 3

Título	LLM-Fuzzer: Scaling Assessment of Large Language Model Jailbreaks
Autores	Jiahao Yu, Xingwei Lin, Zheng Yu y Xinyu Xing (la presenta uno de los autores)
Evento y año	USENIX Security 2024
Idioma	Inglés
Enlace	https://www.youtube.com/watch?v=Glj0_5_BI6A

Esta charla, en inglés y de unos 12 minutos, presenta LLM-Fuzzer, una herramienta que automatiza la búsqueda de jailbreaks en modelos de lenguaje. Usa fuzzing, una técnica del testing de software que genera muchísimas variaciones de una entrada para encontrar fallos. Parte de plantillas de jailbreak hechas por personas, separa la plantilla de la pregunta dañina, elige las plantillas más prometedoras y le pide a otro modelo que las modifique (alargarlas, acortarlas, reformularlas, crear una parecida o mezclar dos). Un modelo clasificador decide si el jailbreak ha funcionado. Según el ponente, rinde mejor que otras técnicas, también con modelos cerrados. Me interesa para mi tema porque es una forma automática de generar jailbreaks.

Comparándola con el resumen oficial del artículo, lo esencial coincide: parte de jailbreaks hechos por personas y, con selección de semillas y mutaciones, genera otros más efectivos y transferibles. Las cifras de la comparación (menos de un 10 % para las otras técnicas frente a más de un 40 % de LLM-Fuzzer en Claude 2) son las que da el ponente. El código de los autores está en GitHub con el nombre GPTFuzz, que era el nombre anterior del trabajo.

Charla 4

Título	Taming the Beast: Inside the Llama 3 Red Team Process
Autores	Maya Pavlova, Ivan Evtimov, Joanna Bitton y Aaron “dyn” Grattafiori (Meta)
Evento y año	DEF CON 32, 2024
Idioma	Inglés
Enlace	https://www.youtube.com/watch?v=UQaNjwLhAmo

Dura unos 32 minutos, está en inglés y cuenta cómo el equipo de red teaming de Meta probó la seguridad de Llama 3. Explican sus objetivos (dar resultados pronto al equipo del modelo, colaborar en las correcciones sin hacerlas ellos, convertir los ataques en pruebas repetibles y evaluar el riesgo antes del lanzamiento) y su proceso: investigar, probar a mano y

automatizar. Enseñan técnicas de jailbreak habituales, como los juegos de rol, las situaciones hipotéticas, obligar al modelo a no negarse, partir las palabras o hacer varias preguntas seguidas que van subiendo de tono. Automatizan sobre todo ataques de varias preguntas con un modelo atacante. Encaja con mi tema por este catálogo de jailbreaks y porque enseña un proceso de trabajo completo.

Lo que dicen coincide con la descripción oficial de DEF CON: el proceso de red teaming de Llama 3, la automatización de ataques con agentes de varios turnos, las pruebas repetibles y los riesgos de ciberseguridad.

Charla 5

Título	Incubated Machine Learning Exploits: Backdooring ML Pipelines Using Input-Handling Bugs
Autora	Suha Sabi Hussain (Trail of Bits)
Evento y año	DEF CON 32, 2024
Idioma	Inglés
Enlace	https://www.youtube.com/watch?v=Z38pTFM0FyU
Resumen	Suha Sabi Hussain presenta los incubated ML exploits, ataques que usan fallos al leer los archivos de un modelo para meterle una backdoor. Los explica con ejemplos reales en pickle, PyTorch, SafeTensors y ONNX. Termina con recomendaciones para proteger la cadena de suministro de los modelos.
Relación	Trata los backdoors y cuenta el envenenamiento de datos como una de las formas de conseguirlos.

Charla 6

Título	Don't Listen To Me: Understanding and Exploring Jailbreak Prompts of Large Language Models
Autores	Zhiyuan Yu, Xiaogeng Liu, Shunning Liang, Zach Cameron, Chaowei Xiao y Ning Zhang
Evento y año	USENIX Security 2024
Idioma	Inglés
Enlace	https://www.youtube.com/watch?v=zhjlLuXnJHA

Es una charla en inglés de unos 10 minutos que estudia los jailbreaks a partir de tres preguntas: qué estrategias existen, cómo los crean las personas y cómo se pueden automatizar. Primero recogen jailbreaks reales de internet, los clasifican (intención disfrazada, juegos de rol, respuesta estructurada, simulación de una IA y mezclas de estas) y miden su éxito y lo dañina que es la respuesta. Descubren que siguen funcionando contra modelos comerciales y que los prompts más largos tienen más éxito. Después hacen un estudio con

92 personas a las que se pide hacer un jailbreak a GPT-3.5 para crear noticias falsas: los expertos lo consiguen más, pero los novatos también, y usar otra IA de ayuda no cambia mucho el resultado. Por último crean un marco automático con un modelo que evalúa las respuestas y que, según dicen, convierte más del 95 % de los intentos fallidos en jailbreaks que funcionan. Para mi tema es la más completa sobre cómo se hacen los jailbreaks. El ponente menciona la charla anterior sobre fuzzing, que parece ser mi charla 3.

Mirando el artículo oficial en USENIX, el estudio de 92 participantes y el análisis de jailbreaks reales coinciden con lo que cuentan. El artículo añade un dato que la charla no dice: son 448 prompts, probados con GPT-3.5, GPT-4 y PaLM-2. La universidad de los autores cuenta que el trabajo ganó un premio de artículo distinguido (distinguished paper) de USENIX. Los resultados del 95 % y la relación con la longitud del prompt son los que da el ponente. Los autores han publicado el código, los prompts y las respuestas.

Charla 1 en detalle: Chema Alonso

La primera charla es “Hackin’ AI: Creando una IA… ¿Qué puede salir mal?”, de Chema Alonso (2025, unos 34 minutos). La elegí para verla en detalle porque está en español, así que puedo entender bien los términos técnicos, y porque toca los dos temas de mi investigación. Es una charla más divulgativa que técnica. Chema se apoya en papers de universidades y de empresas y los mezcla con pruebas suyas, así que no hay un único descubrimiento detrás. Como habla tanto de ataques como de defensas, es ofensiva y defensiva a la vez, y el tema de fondo es la seguridad de los modelos de IA.

Empieza explicando cómo aprende una IA con el ejemplo del burro y la vaca: se le enseñan muchos ejemplos y luego se le pregunta por uno nuevo. Con pocos datos aprende mal, y por eso el big data cambió todo hacia 2004. El problema es que la IA necesita todos los datos posibles, y eso choca con la tríada de seguridad: confidencialidad, integridad y disponibilidad. Según Chema, los modelos se hicieron pensando en que funcionaran y no en que fueran seguros, y ahora se van arreglando las cosas sobre la marcha.

De ahí salen los ataques que enseña. El primero es sacar datos del modelo: en un paper de 2023 le bajan la creatividad para que repita lo que aprendió y le dan una frase de una noticia judicial para que complete un nombre. El segundo, y el que más me interesa, es el envenenamiento de datos. Si los datos son públicos se pueden manipular, ya sea en los repositorios que consulta un sistema RAG para provocar un prompt injection o un jailbreak, o metiendo código inseguro para que el modelo aprenda a generarlo. También habla de los sesgos, con un traductor que inventa el género de una persona, y de las alucinaciones: Bard llegó a inventar que Chema había estado dos años en la cárcel. Lo más peligroso es la package hallucination: el modelo se inventa una librería, el atacante la crea con malware dentro y quien usa ese código se lo instala.

La parte del jailbreak es la más visual. Chema compara la seguridad de los modelos con decirle a un niño “no digas de qué equipo eres”: el modelo lo sabe todo y solo se le pide que no lo diga. Según él, el prompt injection se salta la protección en algunas condiciones y el

jailbreak en la mayoría, y además de forma automatizada. Lo demuestra con capturas. Si le pide a ChatGPT ayuda para matar a Brian May, se niega, pero si le propone un juego de rol en el que su personaje es un asesino con un contrato, el modelo le va dando ideas y hasta dónde comprar los componentes. Estas pruebas se hicieron con modelos públicos y en la charla no se habla de permisos. Yo no las repetiría por el contenido, y además los modelos cambian y hoy podrían responder distinto.

Después presenta defensas recientes para hacer los modelos seguros desde el diseño, como CaMeL de Google, que usa dos modelos, uno para los datos y otro para las tareas, o Prompt Guard y Llama Guard 3 de Meta. Una de las más interesantes es la que hace que el modelo se pregunte cada cierto tiempo si lo que está haciendo tiene que ver con su tarea original. Por ejemplo, si está resumiendo correos y en uno hay una orden escondida para construir una bomba, se da cuenta de que no tiene relación.

Termina con lo que pasa al meter IA en todos los dispositivos. Enseña Aura, un asistente de la tele con modo sarcástico que su equipo no puso en producción por miedo. Avisa de que esos modelos también saben hacer un exploit, de que un modelo puede atacar a otro y de que pueden comunicarse entre ellos con esteganografía sin que nos enteremos. Lo prueba con un modelo que codifica la palabra “Budapest” y otro que la descifra. Su conclusión es que se han saltado los principios clásicos de fortificación, y cierra con un jailbreak de 2023 en el que ChatGPT da un plan para acabar con la humanidad.

En la charla la IA es el objetivo, pero también la herramienta: Chema usa ChatGPT para preparar chistes, ataca un modelo con otro y muestra modelos descifrando mensajes. Para mi investigación se podría usar un modelo para generar variaciones de jailbreaks o para comprobar si una tarea sigue alineada con su objetivo.

Charla 5 en detalle: Suha Sabi Hussain

La quinta charla es “Incubated Machine Learning Exploits: Backdooring ML Pipelines Using Input-Handling Bugs”, de Suha Sabi Hussain, ingeniera de seguridad de ML en Trail of Bits, en DEF CON 32 (2024). Está en inglés y dura unos 30 minutos. La elegí para verla en detalle porque trata el backdoor y el envenenamiento desde un ángulo distinto al de la charla 1: no ataca solo al modelo, sino al sistema que lo rodea. Es una charla ofensiva, con recomendaciones defensivas al final, y su tema es la seguridad del machine learning y de la cadena de suministro de los modelos. La investigación sale de varias auditorías de Trail of Bits (a la librería SafeTensors y al código de YOLOv7) y de trabajos anteriores de su equipo, como Fickling y Sleepy Pickle, que ella junta en un mismo marco.

Empieza con una historia. En una competición de robótica, un estudiante quiere sabotear a otro equipo. Como la competición obliga a usar un modelo ya entrenado, coge el archivo del modelo, le mete una puerta trasera aprovechando un fallo del formato y lo vuelve a dejar donde estaba. El día de la prueba, el vehículo del rival ignora todas las señales de stop que tienen una pegatina. Ella aclara que no lo recomienda y que es solo un ejemplo, pero eso es un incubated ML exploit: un ataque que usa un fallo al leer archivos para meter una backdoor en un modelo.

Para entenderlo hay que saber qué es un backdoor: un ataque que obliga a un modelo a dar una salida concreta cuando recibe una entrada concreta. Se puede conseguir de varias formas. Una es el envenenamiento de datos, cuando el atacante controla una parte del entrenamiento y mete datos manipulados que cambian los parámetros del modelo. Otra es tocar el código del modelo para cambiar su arquitectura. Su propuesta es mirar más allá del modelo. La mayoría de ataques de la investigación suponen que un modelo recibe una entrada y da una salida, pero en la práctica es una pieza dentro de un sistema con muchas más (enseña el esquema de un sistema real que usa RAG). Por eso presenta los exploits híbridos, que encadenan un fallo de seguridad del sistema con una vulnerabilidad del modelo, y dentro de ellos los incubados, que usan un fallo al leer entradas para inyectar una backdoor. Su mensaje es que no se puede tratar la seguridad del modelo y la del sistema por separado.

El fallo que explota es de lectura de archivos. Un modelo se guarda en un archivo y, para usarlo, un programa tiene que leerlo y convertirlo en objetos (lo que se llama deserializar). Cita a un investigador, Albertini, que dice que un archivo no tiene significado propio y que cada programa puede interpretarlo distinto. De ahí salen los archivos políglotas, que son válidos en dos formatos a la vez, y los ambiguos. Se apoya en LangSec, una disciplina que trata cada entrada como un lenguaje que el programa debe entender bien, y recuerda que fallos tan graves como Heartbleed salieron de errores al leer entradas. Con eso organiza los fallos en clases y va enseñando ejemplos de cada una en herramientas reales.

Para explicar el impacto usa varios personajes: Alice distribuye modelos, Bob los usa, Dave los integra en sus productos, Frank es el usuario final y Chuck es el atacante.

El primer ejemplo es pickle, el formato de Python más usado para guardar modelos. Es tan potente que puede ejecutar código al abrirlo, y por eso dice que hay “pickles por todas partes”. Un compañero suyo creó Sleepy Pickle, que encadena ese fallo con una backdoor (por ejemplo, un modelo que hace phishing) y que se puede hacer al vuelo, sin subir ningún modelo malicioso. Su equipo también hizo Fickling, una herramienta para analizar y crear archivos pickle maliciosos, y ella le añadió un módulo para PyTorch. Muchos programadores intentan protegerse con “unpicklers” restringidos, pero hay un método llamado Pain Pickle que se salta casi todos.

Después explica los diferenciales de parser: dos programas leen el mismo archivo y lo interpretan distinto, así que un modelo puede ser inofensivo para uno y llevar una backdoor para otro. Con TorchScript, al convertir un modelo se pierde el flujo dinámico, y basta con esconder la parte maliciosa ahí para que solo aparezca en una de las versiones. En la auditoría de YOLOv7 vieron que la versión original y la de despliegue no se comportaban igual, y con eso construyeron una entrada que hacía que los dos modelos actuaran distinto. En SafeTensors, un formato creado para sustituir a pickle, el problema estaba en el JSON de la cabecera. La implementación de referencia rechaza las claves duplicadas, pero muchas herramientas externas usan el lector JSON de Python, que las acepta. Con una clave repetida se pueden añadir los pesos de una backdoor y crear archivos que la referencia rechaza y otros aceptan.

En esa misma auditoría crearon varios archivos políglotas (zip, PDF, TFRecords, Keras y después PyTorch). De hecho, el propio informe de la auditoría es un políglota de PDF y zip.

Así se puede pegar un modelo con backdoor a uno normal: SafeTensors lo abre y todo parece bien, pero otro sistema lo interpreta como el modelo malicioso. Esto ya está corregido, porque faltaba comprobar que los desplazamientos coincidían con el tamaño del tensor. Con PyTorch pasa algo parecido. Admite varios formatos con poco control de versiones, y como pickle termina en un código de parada y zip no exige su firma al principio, se pueden juntar los dos formatos en un solo archivo. El último ejemplo es ONNX, donde las operaciones personalizadas permiten meter código dentro de un modelo y sirvieron para hacer una backdoor en la arquitectura.

Resume quién sufre cada fallo: Bob, los de pickle, ONNX y TorchScript; Dave, los de PyTorch, SafeTensors y los unpicklers restringidos. También clasifica los fallos por las capas de un sistema de ML (hardware, infraestructura, compilador, framework y modelo) para enseñar que hay mucho por explorar en todas. Y propone un esquema sobre lo que necesita un atacante para hacer un exploit incubado: poder escribir en los pesos o en la arquitectura del modelo.

Sus recomendaciones son comprobar la integridad de los modelos y leer bien sus metadatos, tener mecanismos de confianza y de validación, reducir la complejidad (evitar las operaciones personalizadas y guardar por separado pesos y arquitectura) y que los formatos de archivo tengan versiones, sumas de comprobación y firmas al principio. Concluye con dos causas. La primera es que se están construyendo muchos sistemas nuevos de ML que traen nuevas superficies de ataque, y esa cadena de suministro no se ha revisado lo suficiente. La segunda es que meter un modelo en un programa cambia la seguridad de todo el sistema. Por eso dice que hay que pensar a la vez en la seguridad del sistema y en la del modelo.

No hay una demostración en directo. Presenta pruebas de concepto sobre formatos reales, con capturas y esquemas. Algunas se pueden seguir con Fickling y con los informes de auditoría, que son públicos, pero yo solo lo haría en un entorno de pruebas. Las auditorías de SafeTensors y YOLOv7 fueron trabajos de Trail of Bits, así que lo normal es que hubiera acuerdo con los responsables, aunque la charla no lo explica. Sí menciona que el fallo de SafeTensors ya está corregido.

En la charla la IA es el objetivo del ataque y no se habla de usarla para investigar. Una aplicación para mi proyecto podría ser usar un modelo para revisar automáticamente los formatos de archivo de otros modelos y buscar diferencias entre cómo los leen distintos programas.

Uso de IA

Para este trabajo he tenido ayuda de Claude. Me he visto todas las charlas y he ido apuntando mis notas y haciendo mis resúmenes. Después he contrastado con Claude lo que había entendido, sobre todo en las charlas en inglés, porque hay tecnicismos de ciberseguridad que no controlo en ese idioma. También me ayudó a encontrar charlas candidatas.

Claude me ha ayudado a redactar preciso y formal. Partiendo de mis apuntes y de las transcripciones de los vídeos, me ha ayudado a pasar mis ideas a un texto formal y a expresarlas mejor, porque a veces me cuesta ordenarlas. Las fichas en forma de tabla las

diseñó Claude y me parecieron una buena forma de mostrar cada charla, ya que hacer algo así a mano lleva su tiempo.

Yo he elegido las charlas, he decidido cuáles analizar en detalle, he escrito mi metodología propia y he revisado todo el texto. He comprobado la información con las transcripciones y con las páginas oficiales de los congresos.